Absolutely — below is a **detailed, exam-ready answer** for each question, written in a way that should help you score **full marks**. I've also added **simple diagrams** where they strengthen the explanation.

> Important note for Q3: The question mentions A1, A2, and A\, but the exact definitions of A1 and A2 depend on your class notes. In many AI classes, these are "partial" evaluation strategies that consider only part of the total path information, while A\ uses both path cost and heuristic. I'll explain that standard interpretation clearly. If you want, send your class slide for Q3 and I'll make it exactly match your lecturer's notation.

---

1) Explain the following concepts

(a) Artificial Intelligence

Definition

Artificial Intelligence (AI) is the branch of computer science that aims to build systems capable of performing tasks that normally require human intelligence. These tasks include:

- reasoning
- problem solving
- learning from data
- understanding language
- perception (vision, speech)
- planning and decision-making

Key idea

AI is about designing **intelligent agents** — systems that **perceive their environment** and **take actions** to achieve goals.

Example

- A chess-playing program chooses the best move.
- A self-driving car detects lanes, pedestrians, and traffic signs.
- A chatbot answers questions.
- A recommendation system suggests movies or products.

Main goals of AI

1. **Represent knowledge**
2. **Reason with that knowledge**
3. **Learn from experience**
4. **Plan actions**

5. **Act autonomously**

Types of AI (briefly)

- **Narrow AI**: good at a specific task (e.g., spam detection, face recognition)
- **General AI**: hypothetical systems with human-like general intelligence across many tasks

Simple diagram

```
Environment  --->  Sensors / Input  --->  AI System  --->  Actions / Output
                     (data received)       (reasoning,  (decision taken)
                                           learning,
                                           planning)
```

Short conclusion

Artificial Intelligence is the science and engineering of creating systems that can behave intelligently by learning, reasoning, and making decisions to solve problems.

(b) State Space

Definition

A **state space** is the complete set of all possible states that can occur in a problem, along with the transitions between those states.

In AI search problems, a **state** represents a snapshot of the system at a particular moment.

Why it matters

When solving a problem by search, the AI:

1. starts from an **initial state**
2. applies **operators/actions**
3. moves through the **state space**
4. tries to reach a **goal state**

Example: 8-puzzle

Each arrangement of the tiles is a **state**.\ Moving the blank tile left/right/up/down creates transitions to new states.

Components of a state-space problem

- **Initial state**

- **Set of possible states**
- **Actions/operators**
- **Transition model**
- **Goal test**
- **Path cost**

Diagram

```
[Initial State]
                |
        -----------------
        |               |
     Action1         Action2
        |               |
     [State A]       [State B]
        |               |
     -------         -------
      |     |         |     |
   [S1]   [S2]     [S3]   [Goal]
```

Important note

A **state space** is not just one state; it is the **entire search landscape** of the problem.

Short conclusion

State space is the structured collection of all possible configurations of a problem and the paths connecting them.

(c) Problem of Learning

Definition

The **problem of learning** in AI is the challenge of enabling a machine to improve its performance on a task using experience or data.

In other words, instead of explicitly programming every rule, we allow the system to **learn patterns** from examples.

Formal view

A system learns when its performance on a task improves with experience.

Main questions in learning

A learning system must answer:

1. **What to learn?**

(rules, concepts, decision boundaries, probabilities, policies)

1. **From what data?**

(examples, rewards, observations)

1. **How to represent the learned knowledge?**

(weights, rules, decision trees, neural networks)

1. **How to generalize?**

It should perform well not only on training data, but also on unseen data.

Core difficulty

The major challenge is **generalization**:

- memorizing training data is easy
- learning a rule that works on new data is hard

Typical learning problems

- classification
- regression
- clustering
- reinforcement learning

Example

A system learns to identify emails as **spam** or **not spam** from labeled examples.

Common issues

- insufficient data
- noisy data
- overfitting
- underfitting
- irrelevant features
- computational complexity

Diagram

```
Training Data  --->  Learning Algorithm  --->  Model  --->  Predictions on New Data
```

Short conclusion

The problem of learning is how to design machines that can automatically discover useful patterns from data and use them to make accurate decisions on unseen cases.

---

2) Indicate whether the following statements are True or False

(i) “LISP is a functional programming language.”

Answer: True

Explanation

LISP is widely known as one of the earliest and most important **functional programming languages**.\ Although modern Lisp dialects support multiple paradigms, it is fundamentally associated with:

- function-based computation
- recursion
- symbolic processing
- list manipulation

So for an exam statement like this, the correct answer is **True**.

(ii) “Prolog is a rule-based programming language, and its interpreter is able to answer queries based on facts and rules provided.”

Answer: True

Explanation

Prolog is a **logic programming language** based on:

- **facts**
- **rules**
- **queries**

The Prolog interpreter uses logical inference to answer questions from the knowledge base.

Example:

```
parent(john,mary).
parent(mary,anna).
grandparent(X,Y) :- parent(X,Z), parent(Z,Y).
```

Query:

```
?- grandparent(john,anna).
```

The interpreter answers by reasoning over the facts and rules.\ Therefore the statement is **True**.

(iii) “Breadth-first search and depth-first search often apply heuristics.”

Answer: False

Explanation

Breadth-First Search (BFS) and **Depth-First Search (DFS)** are **uninformed search algorithms**.\ They do **not use heuristics**.

- **BFS** expands the shallowest node first.
- **DFS** expands the deepest node first.

A **heuristic** is an estimate of how close a state is to the goal, and heuristics are used in **informed search** methods such as:

- Greedy Best-First Search
- A* Search

So the statement is **False**.

(iv) “In problem solving by searching, a state is equivalent to a node, i.e., they contain the same information.”

Answer: False

Explanation

A **state** and a **node** are related, but they are **not the same**.

State

A state is a representation of the problem configuration.

Node

A node in a search tree usually contains:

- the **state**
- the **parent node**
- the **action used to reach it**
- the **path cost**
- the **depth**
- sometimes heuristic/evaluation information

So a node contains **more information** than a state.

Example

Two different paths may lead to the **same state**, but these will be represented by **different nodes** because:

- the parent may differ
- the path cost may differ
- the depth may differ

Hence the statement is **False**.

---

3) In problem-solving by searching, we discussed the A1, A2, and A\ algorithm. Explain why A1 and A2 are not optimal, while A\ is optimal.

Core idea

The reason **A1** and **A2** are not optimal is that each of them uses **only partial information** when deciding which node to expand.\ By contrast, **A*** uses the **total estimated solution cost**, which makes it optimal under the proper heuristic conditions.

Standard explanation of A*

A* evaluates a node using:

\[f(n) = g(n) + h(n)]

Where:

- (\mathbf{g(n)}) = actual cost from the start node to node (n)
- (\mathbf{h(n)}) = estimated cost from node (n) to the goal
- (\mathbf{f(n)}) = estimated total solution cost through node (n)

So A* combines:

- **what has already been spent** ((g(n)))
- **what is still expected** ((h(n)))

This balance is the key reason for its optimality.

Why A1 is not optimal

If **A1** considers only one part of the information (for example, only heuristic closeness or only immediate attractiveness), then it can be **misled**.

Typical problem

Suppose a node looks close to the goal according to the heuristic, but the actual path to reach it is very expensive.\ A1 may still choose it because it ignores the full path cost.

Result

It may find **a solution**, but not necessarily the **least-cost solution**.

Why A2 is not optimal

Similarly, if **A2** considers only another partial aspect (for example, path cost so far, depth, or another incomplete measure), then it can ignore how promising the future path is.

Typical problem

A node may have a cheap cost so far, but it may lead to a very expensive remainder.\ If A2 ignores the estimated remaining distance, it may expand nodes that do not lead to the best overall solution.

Result

Again, it may return a valid solution, but not always the best one.

Why A* is optimal

A* is optimal **when the heuristic is admissible** (and, for graph search, preferably consistent).

Admissible heuristic

A heuristic is **admissible** if it **never overestimates** the true cost to the goal:

\[h(n) \leq h^*(n)]

where (h^*(n)) is the true minimum remaining cost.

Why this matters

If (h(n)) never overestimates, then:

\[f(n) = g(n) + h(n)]

never overestimates the true total solution cost of a path through (n).

So A* expands nodes in an order that guarantees it does not skip a cheaper optimal solution in favor of a more expensive one.

Therefore

The first goal node selected by A* is guaranteed to be **optimal**.

Intuitive comparison

```
A1: uses only part of the story  -> can be fooled
A2: uses only another part       -> can be fooled
A*: uses both past cost + future estimate -> best balanced decision
```

Visual intuition

```
Start
        /     \
       /       \
   Cheap so far   Looks close to goal
   but bad later  but expensive path

A1 or A2 may choose one side using incomplete information.
A* considers both:
   f(n) = g(n) + h(n)
 and therefore chooses the path with the lowest total estimated cost.
```

Final exam-ready conclusion

A1 and A2 are not optimal because each uses only incomplete information about the quality of a path, so they can be misled into selecting a suboptimal route. A\, *however, evaluates each node using both the exact cost already incurred and an admissible estimate of the remaining cost, (f(n)=g(n)+h(n)). Because of this, A*\ is guaranteed to return the least-cost solution and is therefore optimal.

> If your class defined A1 and A2 differently, send me that slide and I'll rewrite this answer in your lecturer's exact notation.

---

4) Compare the MP model and the 1st learning machine: Rosenblatt's perceptron. Is there any difference?

Yes, there is a difference.

Both the **McCulloch–Pitts (MP) neuron model** and the **Rosenblatt perceptron** are threshold-based neuron models, but the perceptron is more advanced because it can **learn from data**.

(a) McCulloch–Pitts (MP) model

The MP neuron is a very early mathematical model of a neuron.

Characteristics

- binary inputs (0 or 1)
- fixed weights
- fixed threshold
- binary output
- no learning mechanism
- mainly used to represent logical functions such as AND, OR, NOT

Operation

The neuron computes a weighted sum of the inputs and compares it with a threshold.

\[y = \begin{cases} 1, & \text{if } \sum w_i x_i \ge \theta \\ 0, & \text{otherwise} \end{cases}]

Limitation

The weights and threshold are chosen manually.\ The model does **not update itself from examples**.

(b) Rosenblatt perceptron

The perceptron extends the threshold-neuron idea by introducing **learning**.

Characteristics

- inputs may be binary or numerical
- weights are adjustable
- has a bias/threshold
- learns from labeled examples
- used for classification
- can find a decision boundary automatically for linearly separable data

Learning rule

The perceptron updates weights based on classification error:

$$w_i^{new} = w_i^{old} + \eta (t - y)x_i$$

Where:

- (η) = learning rate
- (t) = target output
- (y) = predicted output

Advantage

Unlike MP neurons, the perceptron **learns from training data**.

Similarities

- both are inspired by biological neurons
- both compute weighted sums
- both use a threshold/activation rule
- both produce a binary decision
- both can solve linearly separable problems

Differences

Aspect	MP Model	Rosenblatt Perceptron
Inputs	Usually binary	Binary or real-valued
Weights	Fixed manually	Learnable
Learning	No	Yes
Purpose	Logical modeling	Pattern classification
Adaptation	Cannot adapt	Adapts from data
Historical role	First formal neuron model	First learning machine

Diagram

```
Inputs x1, x2, ..., xn
        \   |   /
         weighted sum
              |
         threshold test
              |
            output
```

The structure looks similar for both, but the **perceptron updates the weights**, while the **MP model does not**.

Final conclusion

Yes, there is a clear difference. The MP model is a fixed logical neuron with no learning ability, while Rosenblatt's perceptron is the first true learning machine because it can automatically adjust its weights using training examples.

---

5) In class we discussed an approach for solving the XOR problem using a single neuron. Explain why a perceptron with a single neuron can solve a nonlinear problem.

Very important conceptual point

A **standard single-layer perceptron cannot solve XOR in the original input space** because XOR is **not linearly separable**.

So if your class discussed solving XOR with a **single neuron**, then the key idea must be:

> Transform the input space nonlinearly first, so that the problem becomes linearly separable in the new feature space.

This is the correct explanation.

Why XOR is nonlinear

For XOR:

x1	x2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

The positive and negative examples cannot be separated by a single straight line in the ((x_1, x_2)) plane.

XOR plot

```
x2
^
|       (0,1) = 1          (1,1) = 0
|
|
|       (0,0) = 0          (1,0) = 1
+------------------------------------> x1
```

No single line can separate the 1's from the 0's.

So how can a single neuron solve it?

A single neuron can solve a nonlinear problem **if the inputs are transformed into a richer feature space**.

Example transformation

Add a nonlinear feature such as:

\[z = x_1 x_2]

Now instead of representing each example only by ((x_1, x_2)), represent it by ((x_1, x_2, x_1x_2)).

This transformation can make the XOR problem linearly separable in the new space.

Why this works

The perceptron itself still performs a **linear separation** — but it does so in the **transformed feature space**.

That linear boundary in the transformed space corresponds to a **nonlinear boundary in the original space**.

So the perceptron is not magically solving a nonlinear problem by itself. Rather:

- the **feature mapping is nonlinear**
- the perceptron remains linear in the new variables
- the overall input-output relationship becomes nonlinear in the original space

Geometric intuition

Original space

Not separable by one line.

Transformed space

After adding a suitable nonlinear feature, the data can be separated by a plane/hyperplane.

```
Original input space:      After nonlinear transformation:

   not separable           separable
   by a single line        by a single plane/hyperplane
```

Important exam statement

A **single perceptron cannot solve XOR directly** in the original two-dimensional input space.\ However, if the input is **nonlinearly transformed** (for example by adding product terms or polynomial features), the XOR samples become linearly separable in the transformed space, and then a single neuron can classify them correctly.

Final conclusion

A perceptron with a single neuron can solve a nonlinear problem only when the input is first mapped into a nonlinear feature space where the problem becomes linearly separable. Thus, the nonlinearity comes from the feature transformation, not from the basic linear decision mechanism of the perceptron itself.

6) Compare the 3 neural network models discussed in class: a single layer perceptron, the multilayer feed-forward neural network, and the RBF network.

This question needs a structured comparison.

1. Single Layer Perceptron (SLP)

Structure

- one layer of adjustable weights
- no hidden layer
- input directly connected to output neuron(s)

Function

Performs linear classification.

Strengths

- simple
- fast to train
- easy to understand
- works well for linearly separable problems

Weaknesses

- cannot solve nonlinear problems like XOR directly

Decision boundary

- linear (a line, plane, or hyperplane)

Diagram

```
Input Layer  -------->  Output Layer
 x1  x2  x3             y
```

2. Multilayer Feed-Forward Neural Network (MLP)

Structure

- input layer
- one or more hidden layers
- output layer
- connections go forward only (no cycles)

Function

Can model complex nonlinear relationships.

Key feature

The hidden layers and nonlinear activation functions allow the network to learn **nonlinear decision boundaries**.

Learning

Typically trained using **backpropagation** and gradient descent.

Strengths

- solves nonlinear classification problems
- universal function approximator
- very powerful and flexible

Weaknesses

- training can be slower
- can get stuck in local minima
- may overfit
- architecture selection can be difficult

Decision boundary

- nonlinear and highly flexible

Diagram

```
Input Layer      Hidden Layer(s)       Output Layer
 x1   x2   x3   --->  o   o   o   --->  o  --->   y
                      o   o   o
```

3. Radial Basis Function (RBF) Network

Structure

- input layer
- one hidden layer of RBF neurons
- output layer

Key idea

The hidden neurons respond strongly only to inputs near a **center**.\ Each hidden unit uses a radial basis function, often a Gaussian:

\[\phi(x) = \exp\left(-\frac{|x-c|^2}{2\sigma^2}\right)]

Where:

- (c) = center
- (\sigma) = width/spread

Interpretation

An RBF network performs classification based on **local regions** of the input space.

Strengths

- good for interpolation
- often faster training than deep MLPs
- local receptive fields
- can approximate nonlinear functions well

Weaknesses

- choosing centers and widths is important
- may require many hidden units
- performance can drop in high-dimensional data

Decision boundary

- nonlinear, often built from localized regions

Diagram

```
Input Layer   --->   RBF Hidden Units   --->  Output Layer
 x1 x2 x3            [centered bumps]          y
```

Detailed comparison table

Feature	Single Layer Perceptron	Multilayer Feed-Forward NN	RBF Network
Hidden layer	No	Yes	Yes (usually one)
Problem type	Linear	Nonlinear	Nonlinear
Can solve XOR?	No	Yes	Yes
Activation	Threshold / linear-type	Nonlinear activations	Radial basis functions
Training	Simple perceptron rule	Backpropagation	Often two-stage (choose centers, then train output weights)
Decision surface	Linear	Global nonlinear	Local nonlinear
Complexity	Low	Medium to high	Medium
Interpretability	High	Lower	Moderate
Generalization style	Global linear	Global nonlinear	Local response

Conceptual summary

Single Layer Perceptron

Best for simple linearly separable tasks.

Multilayer Feed-Forward Network

Best when the relationship between input and output is complex and nonlinear.

RBF Network

Good when the solution can be formed by combining **local prototypes/regions** of the input space.

Final conclusion

The single-layer perceptron is the simplest model and works only for linearly separable data. The multilayer feed-forward neural network introduces hidden layers and nonlinear activations, making it powerful enough to solve complex nonlinear problems. The RBF network is also nonlinear, but unlike the MLP, it uses localized radial basis functions in the hidden layer, making it especially useful for interpolation and local pattern representation.

---

7) When computing the principal components using PCA, the space can be too large for traditional computers in terms of memory and time. How can one compute it more efficiently while guaranteeing the projection vectors are guaranteed to be those defined by PCA?

Main idea

When the dimensionality of the data is very large, computing PCA through the full covariance matrix can be too expensive.

The efficient and exact solution is to compute PCA using:

- **Singular Value Decomposition (SVD)**, or equivalently
- the **dual / small covariance matrix method** (also called the Gram matrix method)

This gives the **same principal components** as standard PCA.

Why standard PCA becomes expensive

Suppose:

- each data sample has dimension (d)
- there are (n) samples

If (d) is very large, forming the covariance matrix

$$C = \frac{1}{n}XX^T$$

requires a $(d \times d)$ matrix, which may be too large in memory and too expensive to diagonalize.

Efficient exact method: use the smaller matrix

If $(d \gg n)$, instead of diagonalizing (XX^T), diagonalize:

$$G = X^T X$$

which is only $(n \times n)$.

This is much cheaper when the number of samples is smaller than the number of features.

How it works mathematically

Let (X) be the centered data matrix.

Solve the eigenvalue problem for the smaller matrix:

\[X^T X v_i = \lambda_i v_i]

Then the principal directions in the original high-dimensional space are obtained by:

\[u_i = \frac{Xv_i}{\sqrt{\lambda_i}}]

(Up to the appropriate normalization constant depending on convention.)

These vectors (u_i) are exactly the eigenvectors of (XX^T), which are the PCA directions.

So this method gives the **same PCA projection vectors**, not an approximation.

Equivalent viewpoint: SVD

Instead of explicitly forming covariance matrices, perform SVD on the centered data matrix:

\[X = U \Sigma V^T]

Then:

- columns of (U) are the principal directions
- singular values in (\Sigma) determine the explained variance

This is numerically stable and efficient.

Why this guarantees the same PCA result

PCA is fundamentally based on the eigenvectors of the covariance matrix.\ SVD and the dual covariance method compute those same eigenvectors indirectly, but exactly.

So the projection vectors are **guaranteed to be the same vectors defined by PCA**.

Simple diagram

```
Standard PCA:
Data X  ---> form XX^T (huge) ---> eigenvectors

Efficient exact PCA:
```

```
Data X  ---> compute X^T X (small) or SVD(X) ---> recover principal vectors
```

Exam-ready conclusion

When the data space is too large, PCA can be computed efficiently and exactly by applying singular value decomposition (SVD) to the centered data matrix, or equivalently by solving the eigenvalue problem for the smaller matrix (X^T X) instead of the large covariance matrix (XX^T). The principal directions recovered from this method are mathematically identical to those produced by standard PCA, so the resulting projection vectors are guaranteed to be the true PCA vectors.

---

8) What is the Occam's razor principle? When would you use it? What aspects of the learning machine does it address?

Definition

Occam's razor is the principle that, among multiple hypotheses that explain the data equally well, one should prefer the **simplest** hypothesis.

In machine learning language

If two models achieve similar training performance, prefer the one with:

- fewer parameters
- simpler structure
- lower complexity
- less unnecessary detail

Why is it important?

A very complex model may fit the training data perfectly, including noise and accidental patterns.\ This is called **overfitting**.

A simpler model is often more likely to capture the **true underlying pattern** and generalize better to unseen data.

When would you use it?

Occam's razor is used when:

1. several models fit the training data equally well
2. you must choose between a simple and a complex hypothesis
3. you want to improve generalization
4. you want to avoid overfitting
5. you are performing model selection

Examples

- choosing a lower-degree polynomial over a high-degree one
- choosing a neural network with fewer hidden units if performance is similar
- selecting fewer relevant features rather than many unnecessary features
- preferring a simpler rule set over a very complicated one

What aspects of the learning machine does it address?

1. Model complexity

It encourages smaller, simpler models.

2. Generalization ability

It helps the model perform well on unseen data, not just the training set.

3. Overfitting control

It discourages memorizing noise in the data.

4. Bias–variance tradeoff

A simpler model may increase bias slightly, but often reduces variance significantly, improving overall prediction quality.

5. Interpretability

Simpler models are often easier to understand and explain.

6. Regularization

Occam's razor motivates methods like:

- L1 regularization
- L2 regularization
- pruning
- early stopping

- feature selection

Visual intuition

```
Too simple model   -> underfitting
Too complex model  -> overfitting
Appropriate simple model -> better generalization
```

```
Training error: decreases with complexity
Test error: decreases first, then increases if model becomes too complex
```

This is the reason simplicity is often preferred.

Final conclusion

Occam's razor states that when several hypotheses explain the data equally well, the simplest one should be preferred. In machine learning, it is used during model selection to reduce overfitting and improve generalization. It addresses important aspects of the learning machine such as complexity, capacity, generalization performance, regularization, and interpretability.

---

9) "20 points are given to anybody who has finished answering the above 8 questions."

Answer

This means that if you complete Questions **1 to 8**, you automatically receive the **20 points for Question 9**.

So by answering all the previous questions, you should get the full 20 marks for this part.

Short "write-in-exam" summary version

If you want, you can write the long answers above. But if time is limited, here is the short version of the key ideas:

- **AI**: systems that imitate intelligent behavior such as learning, reasoning, and problem solving.

- **State space**: all possible states of a problem and transitions among them.
- **Problem of learning**: enabling a machine to improve performance from data/experience.
- **True/False**:
    - LISP functional language → **True**
    - Prolog rule-based/query answering → **True**
    - BFS/DFS use heuristics → **False**
    - State = node → **False**
- **A1/A2 vs A**: *A1 and A2 use incomplete information; A* uses (f(n)=g(n)+h(n)) and is optimal with an admissible heuristic.
- **MP vs Perceptron**: MP is fixed and does not learn; perceptron learns weights from examples.
- **XOR with one neuron**: not possible in original input space; possible only after nonlinear feature transformation.
- **SLP vs MLP vs RBF**: SLP linear only; MLP nonlinear with hidden layers; RBF nonlinear with localized radial responses.
- **Efficient PCA**: use SVD or eigenvectors of (X^T X) instead of (XX^T); gives exact PCA vectors.
- **Occam's razor**: prefer the simplest model among equally good ones; helps generalization and reduces overfitting.